

Threads Übungen

ThreadUtil

Schreibe eine Methode **sleep** für eine Hilfsklasse **ThreadUtil**, die das Schlafen eines Threads für eine bestimmte Dauer ermöglicht, ohne dass der aufrufende Code die InterruptedException behandeln muss.

Verwende die Klasse `java.time.Duration` als **Parameter**, um die Zeitdauer anzugeben.

Die Methode `sleep` soll intern `Thread.sleep(...)` verwenden, dabei aber die Checked Exception (InterruptedException) in eine Unchecked Exception umwandeln.

Beispielverwendung:

```
public static void main(String[] args) {  
    ThreadUtil.sleep(Duration.ofSeconds(1)); // no try-catch required.  
}
```

ThreadUtil: Lösung

```
final class ThreadUtil {  
  
    public static void sleep(Duration duration) {  
        try {  
            Thread.sleep(duration.toMillis()); // Thread::sleep expects a value in ms  
        } catch (InterruptedException e) {  
            throw new RuntimeException(e);  
        }  
    }  
  
    public static void main(String[] args) throws InterruptedException {  
        ThreadUtil.sleep(Duration.ofSeconds(1));  
    }  
}
```

Dog Eat Cat

Schreibe eine App, die folgende drei Threads startet:

1. Gibt für immer "Dog " jede Sekunde auf der Konsole aus.
2. Gibt für immer "Eat " jede Sekunde auf der Konsole aus.
3. Gibt für immer "Cat " jede Sekunde auf der Konsole aus.

Dog Eat Cat: Lösung

```
class DogEatCat {  
  
    private static final int ONE_SECOND = 1000;  
  
    public static void main(String[] args) {  
        new Thread(() -> println("Dog")).start();  
        new Thread(() -> println("Eat")).start();  
        new Thread(() -> println("Cat")).start();  
    }  
  
    private static void println(String message) {  
        while (true) {  
            System.out.print(message);  
            System.out.print(" ");  
            ThreadUtil.sleep(Duration.ofSeconds(1));  
        }  
    }  
}
```

BTW: Die Konsole in Java ist thread-safe, daher gibt es keine Probleme, wenn mehrere Threads gleichzeitig versuchen, etwas in die Konsole zu schreiben.

StringAdder

Schreibe zwei Threads **A** und **B**.

A und B schreiben in **dieselbe** Liste.

A fügt **x-Mal (limit) "a"** zur Liste hinzu. **Nach jedem Hinzufügen soll 10 ms gewartet werden.**

B fügt **x-Mal (limit) "b"** zur Liste hinzu. **Nach jedem Hinzufügen soll 10 ms gewartet werden.**

Starte A und B als Threads in folgender Methode:

`Thread.sleep(10);`

`static List<String> compute(int limit) throws InterruptedException` *// returns the result of the shared list*

Schreibe einen **Unit Test**, der das Ergebnis von **compute(10)** testet.

```
class ListThreads {  
  
    static List<String> compute(int limit) throws InterruptedException {  
        List<String> sharedList;  
        // TODO: erzeuge Threads für A und B und starte Threads, die beide die sharedList befüllen.  
        return sharedList;  
    }  
}
```

StringAdder: Lösung

```
class StringAdderRunnable {  
  
    static List<String> compute(int limit) throws InterruptedException {  
        // var sharedList = new LinkedList<String>(); // Does not work!  
        var sharedList = Collections.synchronizedList(new LinkedList<String>());  
  
        var aThread = new Thread(new AAdder(sharedList, limit));  
        var bThread = new Thread(new BAdder(sharedList, limit));  
  
        aThread.start();  
        bThread.start();  
  
        aThread.join();  
        bThread.join();  
  
        return sharedList;  
    }  
  
    public static void main(String[] args) throws InterruptedException {  
        System.out.println(compute(10));  
    }  
}
```

FYI: strings ist hier eine **mutable** shared list.
Problem: run() gibt void zurück, d. h. es gibt kein Rückgabewert.

```
class AAdder implements Runnable {  
    private final int limit;  
    private final Collection<String> strings;  
  
    public AAdder(Collection<String> strings, int limit) {  
        this.strings = strings;  
        this.limit = limit;  
    }  
  
    @Override  
    public void run() {  
        for (int i = 0; i < limit; i++) {  
            strings.add("a");  
            ThreadUtil.sleep(Duration.ofMillis(10));  
        }  
    }  
}
```

```
class BAdder implements Runnable {  
    private final int limit;  
    private final Collection<String> strings;  
  
    public BAdder(Collection<String> strings, int limit) {  
        this.strings = strings;  
        this.limit = limit;  
    }  
  
    @Override  
    public void run() {  
        for (int i = 0; i < limit; i++) {  
            strings.add("b");  
            ThreadUtil.sleep(Duration.ofMillis(10));  
        }  
    }  
}
```

Don't repeat yourself...



StringAdder: Lösung (DRY)

```
class StringAdderOneClass {  
  
    private static final int TEN_TIMES = 10;  
  
    static List<String> compute() throws InterruptedException {  
        // var sharedList = new LinkedList<String>(); // Does not work!  
        var sharedList = Collections.synchronizedList(new LinkedList<String>());  
  
        var aThread = new Thread(new StringAdder(sharedList, "a", TEN_TIMES));  
        var bThread = new Thread(new StringAdder(sharedList, "b", TEN_TIMES));  
  
        aThread.start();  
        bThread.start();  
  
        aThread.join();  
        bThread.join();  
  
        return sharedList;  
    }  
  
    public static void main(String[] args) throws InterruptedException {  
        System.out.println(compute(10));  
    }  
}
```

```
class StringAdder implements Runnable {  
  
    private final Collection<String> strings;  
    private final String string;  
    private final int limit;  
  
    StringAdder(Collection<String> strings, String string, int limit) {  
        this.strings = strings;  
        this.string = string;  
        this.limit = limit;  
    }  
  
    @Override  
    public void run() {  
        for (int i = 0; i < limit; i++) {  
            // synchronized (characters) { // alternative, if strings is not synchronized  
            strings.add(string); // bad smell: parameters should not be modified  
            // }  
            ThreadUtil.sleep(Duration.ofMillis(10));  
        }  
    }  
}
```

FYI: strings ist hier eine **mutable** shared list.
Problem: run() gibt void zurück, d. h. es gibt kein Rückgabewert.

StringAdder: Lösung (DRY und Lambda)

```
class StringAdderLambda {  
  
    static List<String> compute(int limit) throws InterruptedException {  
        // var sharedList = new LinkedList<String>(); // works with synchronized  
        var sharedList = Collections.synchronizedList(new LinkedList<String>());  
  
        var aThread = new Thread(stringAdder(sharedList, "a", limit));  
        var bThread = new Thread(stringAdder(sharedList, "b", limit));  
  
        aThread.start();  
        bThread.start();  
  
        aThread.join();  
        bThread.join();  
  
        return sharedList;  
    }  
  
    private static Runnable stringAdder(Collection<String> sharedList, String string, int limit) {  
        return () -> {  
            for (int i = 0; i < limit; i++) {  
                sharedList.add(string); // bad smell: parameters should not be modified  
                ThreadUtil.sleep(Duration.ofMillis(10));  
            }  
        };  
    }  
}
```

FYI: Kein guter Stil, da der Parameter hier modifiziert wird.
Problem: run() gibt void zurück, d. h. es gibt kein Rückgabewert.

StringAdder: Lösung (DRY und SyncLambda)

```
class StringAdderLambda {  
  
    static List<String> compute(int limit) throws InterruptedException {  
        var sharedList = new LinkedList<String>(); // works with synchronized  
        // var sharedList = Collections.synchronizedList(new LinkedList<String>());  
  
        var aThread = new Thread(stringAdder(sharedList, "a", limit));  
        var bThread = new Thread(stringAdder(sharedList, "b", limit));  
  
        aThread.start();  
        bThread.start();  
  
        aThread.join();  
        bThread.join();  
  
        return sharedList;  
    }  
  
    private static Runnable stringAdder(Collection<String> sharedList, String string, int limit) {  
        return () -> {  
            for (int i = 0; i < limit; i++) {  
                synchronized (sharedList) { // bad smell: parameters should not be used as mutex because you cannot ensure here, that always the same list is passed.  
                    sharedList.add(string); // bad smell: parameters should not be modified  
                }  
                ThreadUtil.sleep(Duration.ofMillis(10));  
            }  
        };  
    }  
}
```

FYI: Kein guter Stil, da der Parameter hier modifiziert wird.
Problem: run() gibt void zurück, d. h. es gibt kein Rückgabewert.

StringAdder: Lösung (Unit Test)

```
class StringAdderTest {
```

```
    @Test
```

```
    void compute() throws InterruptedException {
```

```
        final int tenTimes = 10;
```

```
        var expected = Stream.of(
```

```
            Stream.generate(() -> "a").limit(tenTimes),
```

```
            Stream.generate(() -> "b").limit(tenTimes)
```

```
        ).flatMap(e -> e).toList();
```

```
        var actual = StringAdder.compute(tenTimes);
```

```
        assertEqualsIgnoreOrder(expected, actual);
```

```
    }
```

```
    private static <T extends Comparable<T>> void assertEqualsIgnoreOrder(Collection<T> expected, Collection<T> actual) {
```

```
        assertEquals(sorted(expected), sorted(actual));
```

```
    }
```

```
    private static <T extends Comparable<T>> List<T> sorted(Collection<T> collection) {
```

```
        return collection.stream().sorted().toList();
```

```
    }
```

```
}
```

Da die Reihenfolge nicht-deterministisch ist, bleibt uns nichts anderes übrig, als die Elemente beider Listen zu sortieren.

Alternativ könnte man beide Listen in ein HashSet umwandeln, da Set die Reihenfolge ignoriert, allerdings würden wir dann nicht mehr testen, ob dieselbe Anzahl an Elementen vorhanden ist.

ThreadSafeList

Gegeben ist folgender Code.

Ändere nur die Klasse ThreadSafeList, sodass der Unit Test "grün" wird.

Du muss **synchronized** verwenden!

```
class ThreadSafeList<E> extends AbstractList<E> {

    private final List<E> elements = new LinkedList<>();

    @Override
    public boolean add(E element) {
        elements.addLast(element);
        return true;
    }

    @Override
    public int size() {
        return elements.size();
    }

    @Override
    public E get(int index) {
        return elements.get(index);
    }

    @Override
    public String toString() {
        return "ThreadSafeList[" + elements + "]";
    }
}
```

Read-only

```
class FixThreadSafeListTest {

    @RepeatedTest(5)
    void readWriteListShouldNotThrowAnyException() throws InterruptedException {
        boolean exceptionWasThrown = FixThreadSafeList.readWriteList();
        assertFalse(exceptionWasThrown);
    }
}
```

Read-only

```
class FixThreadSafeList {
    private static final int TEN_TIMES = 10;

    static boolean readWriteList() throws InterruptedException {
        List<Integer> threadSafeList = new ThreadSafeList<>();
        threadSafeList.add(0);

        boolean[] exceptionWasThrown = {false};

        var aThread = new Thread(() -> {
            for (int i = 1; i <= TEN_TIMES; i++) {
                try {
                    threadSafeList.add(i);
                } catch (Exception e) {
                    e.printStackTrace();
                    exceptionWasThrown[0] = true;
                }
                try { Thread.sleep(10); } catch (InterruptedException e) { throw new RuntimeException(e); }
            }
        });

        var bThread = new Thread(() -> {
            for (int i = 1; i <= TEN_TIMES; i++) {
                try {
                    System.out.println(threadSafeList);
                } catch (Exception e) {
                    e.printStackTrace();
                    exceptionWasThrown[0] = true;
                }
                try { Thread.sleep(10); } catch (InterruptedException e) { throw new RuntimeException(e); }
            }
        });

        aThread.start();
        bThread.start();

        aThread.join();
        bThread.join();

        return exceptionWasThrown[0];
    }

    public static void main(String[] args) throws InterruptedException {
        readWriteList();
    }
}
```

ThreadSafeList: Lösung

```
class ThreadSafeList<E> extends AbstractList<E> {  
  
    private final List<E> elements = new LinkedList<>();  
  
    @Override  
    public synchronized boolean add(E element) {  
        elements.addLast(element);  
        return true;  
    }  
  
    @Override  
    public synchronized int size() {  
        return elements.size();  
    }  
  
    @Override  
    public synchronized E get(int index) {  
        return elements.get(index);  
    }  
  
    @Override  
    public synchronized String toString() {  
        return "ThreadSafeList[" + elements + "];"  
    }  
}
```

Die Methode toString() muss auch synchronized werden, da die Liste nicht gleichzeitig geschrieben und gelesen werden kann.

Das liegt daran, dass System.out.println(threadSafeList) über alle Elemente in threadSafeList iteriert. Wenn sich aber währenddessen die Anzahl der Elemente ändert, kommt es zu einer ConcurrentModificationException.

FYI: Um ThreadSafeList 100% thread-safe zu machen, müssten alle Methoden aus AbstractList<E> überschrieben und geschützt werden.

SynchronizedArrayList

Ändere die folgende Klasse so, dass sie thread-safe ist.

```
/**
 * Change this class so that it is thread-safe.
 */
class SynchronizedArrayList<T> {

    private final ArrayList<T> elements = new ArrayList<>();

    public boolean add(T element) {
        return elements.add(element);
    }

    public int size() {
        return elements.size();
    }
}
```

SynchronizedArrayList: Lösung

```
class SynchronizedArrayList1<T> {  
  
    private final List<T> elements = Collections.synchronizedList(new ArrayList<>());  
  
    public boolean add(T element) {  
        return elements.add(element);  
    }  
  
    public int size() {  
        return elements.size();  
    }  
}
```

```
class SynchronizedArrayList2<T> {  
  
    private final List<T> elements = new ArrayList<>();  
  
    public boolean add(T element) {  
        synchronized (elements) {  
            return elements.add(element);  
        }  
    }  
  
    public synchronized int size() {  
        synchronized (elements) {  
            return elements.size();  
        }  
    }  
}
```

Drone

Gegeben ist folgender Code.

Ändere ihn, sodass er thread-safe ist.

```
final class App {  
  
    private static final int NUMBER_OF_DRONES = 100;  
  
    public static void main(String[] args) throws InterruptedException {  
        try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {  
            ChargingStation centralOne = new ChargingStation();  
            for (int id = 1; id <= NUMBER_OF_DRONES; id++) {  
                executor.submit(new Drone(id, centralOne));  
            }  
            executor.awaitTermination(Long.MAX_VALUE, TimeUnit.SECONDS);  
        }  
    }  
}
```

```
class ChargingStation {  
  
    private int numberOfLandedDrones = 0;  
  
    boolean isOccupied() {  
        return numberOfLandedDrones != 0;  
    }  
  
    void land() {  
        numberOfLandedDrones++;  
        checkNumberOfLandedDrones();  
    }  
  
    void takeOff() {  
        numberOfLandedDrones--;  
        checkNumberOfLandedDrones();  
    }  
  
    private void checkNumberOfLandedDrones() {  
        if (numberOfLandedDrones < 0 || numberOfLandedDrones > 1) {  
            System.err.printf("numberOfLandedDrones must be between 0 and 1 but was %d%n", numberOfLandedDrones);  
            System.exit(404);  
        }  
    }  
}
```

```
class Drone implements Runnable {  
  
    private final int id;  
    private final ChargingStation chargingStation;  
  
    private int battery = 100;  
  
    Drone(int id, ChargingStation chargingStation) {  
        this.id = id;  
        this.chargingStation = chargingStation;  
    }  
  
    void fly() {  
        battery -= 10;  
        if (battery <= 10) {  
            emergencyLand();  
        } else if (battery <= 30) {  
            tryLand();  
        }  
    }  
  
    void tryLand() {  
        if (!chargingStation.isOccupied()) {  
            chargingStation.land();  
            recharge();  
            chargingStation.takeOff();  
        }  
    }  
  
    private void emergencyLand() {  
        battery = 0;  
    }  
  
    void recharge() {  
        battery = 100;  
    }  
  
    @Override  
    public void run() {  
        while (battery > 0) {  
            fly();  
        }  
    }  
  
    @Override  
    public String toString() {  
        return "%s (%d)".formatted(id, battery);  
    }  
}
```


Drone: Lösung

```
class Drone implements Runnable {

    private final int id;
    private final ChargingStation chargingStation;

    private int battery = 100;

    Drone(int id, ChargingStation chargingStation) {
        this.id = id;
        this.chargingStation = chargingStation;
    }

    void fly() {
        battery -= 10;
        if (battery <= 10) {
            emergencyLand();
        } else if (battery <= 30) {
            tryLand();
        }
    }

    void tryLand() {
        synchronized (chargingStation) {
            if (!chargingStation.isOccupied()) {
                chargingStation.land();
                recharge();
                chargingStation.takeOff();
            }
        }
    }

    private void emergencyLand() {
        battery = 0;
    }

    void recharge() {
        battery = 100;
    }

    @Override
    public void run() {
        while (battery > 0) {
            fly();
        }
    }

    @Override
    public String toString() {
        return "%s (%d)".formatted(id, battery);
    }
}
```